claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a4f40a18b014aae01.jsonl`
- SHA-256: `ed2b4e3ed8f9947ac4a6c44ae6985949a5e734aac01ed2d82bdb84aa0694b514`
- Source modified: `2026-06-13T19:15:51+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:14:49.684Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"Two different 'min_players' ? persisted in_court_ratio always uses the literal default 2, ignoring config fusion.min_players","severity":"low","file":"backend/pipeline/segmenters/fusion_hybrid.py:106, backend/pipeline/detectors/fusion_features.py:99,200,208","detail":"There are two distinct `min_players`: (a) the served Gate-B threshold `FusionConfig.min_players` (used only in `_select_for_handoff`), and (b) the `in_court_ratio` parameter (default 2) baked into `compute_person_features`. `_person_features` calls `ff.compute_person_features(...)` WITHOUT forwarding any min_players, so the persisted `in_court_ratio` feature is always computed at the hardcoded threshold 2, regardless of `indexing.fusion.min_players`. This is defensible (the persisted feature is meant to be a fixed, config-independent column for the learned fusion in P3), and the gate threshold legitimately lives elsewhere ? but the name collision invites a future reader to assume changing `fusion.min_players` also retunes the persisted `in_court_ratio`, which it does not. No functional bug today.","fix":"Optional: add a one-line comment at fusion_hybrid.py:106 noting that in_court_ratio's threshold is intentionally the fixed feature-column default (2) and is independent of the served Gate-B `fusion.min_players`, so the two are not conflated later."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:14:52.541Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:14:53.508Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
```

```
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13        never imports torch / touches the GPU.
14      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17        **Persisted candidates remain the full superset** regardless, so the offline
experiment
18        harness can re-score any variant.
19
20      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
23      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24      experiment harness (EXPERIMENT_HARNESS.md), never silently.
25      """
26      from typing import Any, Dict, List, Optional, Tuple
27
28      from backend.pipeline.segmenters.base import segmenter_registry
29      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30      from backend.pipeline.detectors.base import DetectorRunner
31      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33      from backend.pipeline.detectors import rally_gate
34      from backend.pipeline.detectors import fusion_features as ff
35
36
37      class FusionSegmenter(TrajectoryHybridSegmenter):
38          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
40          family = "fusion"
41          display_label = "Fusion"
42
43          def __init__(self, db: Any, config: Any):
44              super().__init__(db, config)
45              # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46              self._person: Optional[Any] = None
47
48          @property
49          def name(self) -> str:
50              return "fusion_hybrid"
51
52          @property
53          def description(self) -> str:
54              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                      "AI provider sets semantic boundaries.")
57
```

```
    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
        if substrate == "tracknet":
            cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
            return TrackNetRunner(cfg), {
                "weights_path": cfg.weights_path,
                "queue_length": cfg.queue_length,
                "fusion_substrate": "tracknet",
            }
        wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
        return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate": "wasb"}

    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                frame_width: float, video_path: str,
                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
        stats = super()._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
        # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always persisted
        # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-estimated
        # over the whole trajectory by rally_gate (camera-agnostic).
        gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps, min_crossings=0)
        stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0

        fcfg = idx_cfg.get("fusion", {})
        if fcfg.get("cue_flags", {}).get("person", False):
            stats.update(self._person_features(cw, points, fps, frame_width, video_path, fcfg))
        return stats

    def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                         frame_width: float, video_path: str,
                         fcfg: Dict[str, Any]) -> Dict[str, float]:
        """GPU path (person cue ON): run the person detector over this window's ORIGINAL-video
        frame range and compute the scale/translation-invariant person features. Lazily builds
        ONE PersonDetector. ? real-video verification is owner-side; this wiring is mock-tested."""
        # Lazy import so the default (person-OFF) path never pulls torch/cv2 through this module.
        from backend.pipeline.detectors.person_detector import PersonDetector
        if self._person is None:
            self._person = PersonDetector(self.config)

        frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
        start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
        det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
        fw = det["frame_width"] or frame_width
        fh = det["frame_height"] or 0.0

        poly = fcfg.get("court_polygon")
        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
                                          court_polygon=polygon, net=net)

    def _select_for_handoff(self, windows: List[Dict[str, Any]],
                            window_stats: List[Dict[str, Any]],
                            idx_cfg: Dict[str, Any]) -> List[int]:
```

```
112             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113             (0) keep every window ? identical to the substrate. Persisted candidates are
114             untouched (full superset)."""
115             fcfg = idx_cfg.get("fusion", {})
116             min_crossings = int(fcfg.get("min_crossings", 0))
117             min_players = int(fcfg.get("min_players", 0))
118             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120             keep: List[int] = []
121             for i, st in enumerate(window_stats):
122                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                     continue
124                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                     continue
126                 keep.append(i)
127             return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131
```

4. user (2026-06-13T19:14:53.648Z)

```
1       """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3       NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4       math is unit-testable in a GPU-free dev container. The detection (person boxes,
5       shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6       module only turns those signals into a handful of **scale/translation-invariant**
7       features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8       memorise this court/camera and fail to generalise off a small labelled set).
9
10      Conventions:
11      - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12      - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14      - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15      - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17      - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19      Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20      floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21      """
22      from __future__ import annotations
23
24      from typing import Dict, List, Optional, Sequence, Tuple
25
26      BBox = Tuple[float, float, float, float]
27      Net = Tuple[str, float]
28
29
30      def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31          """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32          (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
```

```
        if polygon is None or len(polygon) < 3:
            return False
        x, y = point
        inside = False
        n = len(polygon)
        j = n - 1
        for i in range(n):
            xi, yi = polygon[i]
            xj, yj = polygon[j]
            # Does the horizontal ray at y cross edge (i, j)?
            if (yi > y) != (yj > y):
                x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
                if x < x_cross:
                    inside = not inside
            j = i
        return inside


    def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        """Normalised foot point (bottom-centre) of a person box ? where the player stands,
        the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, _y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)


    def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)


    def person_in_court(box: BBox, frame_width: float, frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
        """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
        True (count everyone) when it is None."""
        if court_polygon is None:
            return True
        return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)


    def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                        frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
        """Per-frame count of in-court persons (one entry per sampled frame)."""
        return [
            sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
            for _frame_idx, dets in sorted(per_frame.items())
        ]


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))


```

```
    def players_in_court(counts: Sequence[int]) -> float:
        """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
        return _median(list(counts))


    def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
        """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
        if not counts:
            return 0.0
        return sum(1 for c in counts if c >= min_players) / len(counts)


    def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
                           frame_width: float, frame_height: float) -> float:
        """Mean per-track centre displacement between consecutive sampled frames, normalised
        by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
        if frame_width <= 0 or len(per_frame) < 2:
            return 0.0
        frames = sorted(per_frame.keys())
        steps: List[float] = []
        for a, b in zip(frames, frames[1:]):
            prev = {tid: box for tid, box in per_frame[a]}
            for tid, box in per_frame[b]:
                if tid in prev:
                    cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
                    cx1, cy1 = _center_norm(box, frame_width, frame_height)
                    steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
        if not steps:
            return 0.0
        return sum(steps) / len(steps)


    def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                         frame_height: float, net: Optional[Net],
                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
        """Fraction of sampled frames with >=1 in-court person on BOTH net halves.

        A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
        The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
        """
        if net is None or not per_frame:
            return 0.0
        axis, pos = net
        both = 0
        for _frame_idx, dets in per_frame.items():
            near, far = False, False
            for _tid, box in dets:
                if not person_in_court(box, frame_width, frame_height, court_polygon):
                    continue
                fx, fy = _foot_norm(box, frame_width, frame_height)
                coord = fx if axis == "x" else fy
                if coord < pos:
                    near = True
                else:
                    far = True
            if near and far:
                both += 1
        return both / len(per_frame)


    def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
                        pad_frac: float) -> bool:
```

```
156         """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
157         width/height (the 'adjacent / in-hand' tolerance)?"""
158         x1, y1, x2, y2 = box
159         px = pad_frac * frame_width
160         py = pad_frac * frame_height
161         return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
162 
163 
164     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                                   frame_width: float, frame_height: float,
166                                   pad_frac: float = 0.03) -> float:
167         """Fraction of (visible shuttle frames that were also person-sampled) in which the
168         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
169         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
170         """
171         if not per_frame or frame_width <= 0 or frame_height <= 0:
172             return 0.0
173         aligned = 0
174         colocated = 0
175         for p in shuttle_points or []:
176             if not getattr(p, "visible", False):
177                 continue
178             dets = per_frame.get(int(p.frame))
179             if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
180                 continue
181             aligned += 1
182             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
183                    for _tid, box in dets):
184                 colocated += 1
185         return colocated / aligned if aligned else 0.0
186 
187 
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194 
195 
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                 frame_width: float, frame_height: float,
198                                 court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                 net: Optional[Net] = None,
200                                 min_players: int = 2) -> Dict[str, float]:
201         """Aggregate the per-window person features from per-frame detections + the in-
window
202         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204         return {
205             "players_in_court": players_in_court(counts),
206             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208             "in_court_ratio": in_court_ratio(counts, min_players),
209             "shuttle_player_colocation": shuttle_player_colocation(
210                 per_frame, shuttle_points, frame_width, frame_height),
211         }
212 
```

5. assistant (2026-06-13T19:14:59.095Z)

Both my hypotheses need checking. Let me verify the key claims in the finding by looking at the config model and the parent class to understand the relationship between the two `min_players`.

6. user (2026-06-13T19:15:00.705Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index bb56881..7f8191c 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -75,6 +75,39 @@ class WasbIndexConfig(BaseModel):
     min_free_gb: float = 0.0


+class FusionConfig(BaseModel):
+    """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
+
+    Every default reproduces control behaviour: the fusion segmenter persists the
+    shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
+    true ? the person-cue features, but it does NOT gate the served handoff unless a
+    threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
+    is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-only);
+    supplied ? off-court persons (spectators / adjacent courts) are excluded.
+    """
+    # Which trajectory substrate seeds the windows (shuttle stays primary).
+    substrate: Literal["wasb", "tracknet"] = "wasb"
+    # Windowing velocity threshold for the fusion family (the engine reads
+    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
+    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
+    velocity_threshold: float = 0.02
+    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
+    # enabled ? so the default path never imports torch / touches the GPU.
+    cue_flags: dict[str, bool] = Field(default_factory=dict)
+    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
+    # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-
scoring).
+    min_crossings: int = Field(default=0, ge=0)
+    # Served Gate B: when the person cue is on, drop windows with median in-court players
+    # < this. 0 = OFF.
+    min_players: int = Field(default=0, ge=0)
+    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
+    court_polygon: Optional[list[list[float]]] = None
+    # Net line for the person two-sided-motion split (person features only ? the shuttle
+    # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
+    net_axis: Literal["x", "y"] = "y"
+    net_position: float = Field(default=0.5, ge=0.0, le=1.0)
+
+
 class IndexingConfig(BaseModel):
     default_segmenter: str = "yolo_hybrid"
     use_gpu: bool = True
@@ -106,6 +139,7 @@ class IndexingConfig(BaseModel):

     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
+    fusion: FusionConfig = Field(default_factory=FusionConfig)

     @field_validator("default_segmenter")
     @classmethod
```

7. user (2026-06-13T19:15:01.230Z)

```
docs\CODE_MAP.md:148:[Omitted long matching line]
backend\config\models.py:84:    threshold is raised (``min_crossings``/``min_players`` default 0
= OFF). The court mask
backend\config\models.py:102:    min_players: int = Field(default=0, ge=0)
```

```
tests\test_config.py:35:    assert f.min_players == 0           # Gate B OFF
backend\eval\experiment.py:91:      - ``min_players`` ? decision-gate **Gate B** (the future
person cue): keep windows
backend\eval\experiment.py:109:    min_players: int = 0
backend\eval\experiment.py:126:            "min_players": self.min_players,
backend\eval\experiment.py:197:    if variant.min_players > 0:
backend\eval\experiment.py:199:        mask = [m and (col[i] >= variant.min_players) for i, m in
enumerate(mask)]
tests\test_experiment.py:82:                        min_crossings=0, min_players=0)
tests\test_experiment.py:107:    preds = predict(Variant(name="gateB", min_players=2), vc)
backend\pipeline\segmenters\fusion_hybrid.py:15:  (and, with the person cue, ``min_players``)
are raised above 0, sub-threshold windows are
backend\pipeline\segmenters\fusion_hybrid.py:117:        min_players =
int(fcfg.get("min_players", 0))
backend\pipeline\segmenters\fusion_hybrid.py:124:            if person_on and min_players > 0
and st.get("players_in_court", 0.0) < min_players:
tests\test_fusion_features.py:60:    assert ff.in_court_ratio(counts, min_players=2) ==
pytest.approx(0.25)
tests\test_fusion_features.py:61:    assert ff.in_court_ratio(counts, min_players=1) ==
pytest.approx(0.75)
backend\pipeline\detectors\fusion_features.py:99:def in_court_ratio(counts: Sequence[int],
min_players: int = 2) -> float:
backend\pipeline\detectors\fusion_features.py:100:    """Fraction of sampled frames with >=
`min_players` in court (track stability vs flicker)."""
backend\pipeline\detectors\fusion_features.py:103:    return sum(1 for c in counts if c >=
min_players) / len(counts)
backend\pipeline\detectors\fusion_features.py:200:                            min_players: int =
2) -> Dict[str, float]:
backend\pipeline\detectors\fusion_features.py:208:        "in_court_ratio":
in_court_ratio(counts, min_players),
```

8. assistant (2026-06-13T19:15:15.225Z)

Now I have the full picture. Let me verify the key claims of the finding precisely.

The finding states:
1. There are two distinct `min_players`: (a) the served Gate-B threshold `FusionConfig.min_players` (used in `_select_for_handoff`), and (b) the `in_court_ratio` parameter (default 2) in `compute_person_features`.
2. `_person_features` calls `ff.compute_person_features(...)` WITHOUT forwarding any `min_players`, so the persisted `in_court_ratio` always uses the hardcoded default 2.

Let me verify each precisely against the code.

Claim 1 ? two distinct min_players:
- `FusionConfig.min_players` (config/models.py:102, default 0) ? used in `_select_for_handoff` at fusion_hybrid.py:117 and :124. CONFIRMED. Note: Gate B at line :124 compares against `players_in_court`, NOT `in_court_ratio`.
- `in_court_ratio(counts, min_players=2)` (fusion_features.py:99) and `compute_person_features(..., min_players=2)` (fusion_features.py:200). CONFIRMED ? default 2.

Claim 2 ? `_person_features` does not forward `min_players`:

Looking at fusion_hybrid.py:106-107:
```python
return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
                                  court_polygon=polygon, net=net)
```
It passes `court_polygon` and `net` but NOT `min_players`. So `compute_person_features` uses its default `min_players=2`, which flows to `in_court_ratio(counts, 2)`. CONFIRMED ? the persisted `in_court_ratio` is always computed at threshold 2, independent of `FusionConfig.min_players`.

Now let me adversarially test whether this is actually a **bug** vs. a benign name-collision/documentation nitpick, which is what the finding itself concedes ("No functional bug today", severity "low").

Key question: Does the served Gate B (`_select_for_handoff`) ever use `in_court_ratio`? Looking at line :124, Gate B gates on `players_in_court`, not `in_court_ratio`. So `FusionConfig.min_players` controls the `players_in_court` threshold for the *served* gate, while `in_court_ratio` is a separate persisted feature column whose own internal `min_players=2` is a fixed feature definition. These genuinely are independent by design ? the persisted feature is meant to be a config-independent column for the learned fusion (P3).

Let me confirm `in_court_ratio` is not consumed anywhere in the served gating path or experiment harness in a way that would make the collision a real bug.

9. user (2026-06-13T19:15:15.817Z)

```
docs\EXPERIMENT_HARNESS.md:175:   `in_court_ratio`, `shuttle_player_colocation`; shuttle-state:
`net_crossings`, `direction_reversals`;
docs\MULTI_SIGNAL_FUSION_PLAN.md:138:| `in_court_ratio` | fraction of frames with ?2 in-court
players | track stability vs flicker |
docs\QUALITY_ITERATIONS.md:66:mean_player_motion, in_court_ratio, shuttle_player_colocation`)
persisted into candidate `stats` for P3's
tests\test_fusion_features.py:60:    assert ff.in_court_ratio(counts, min_players=2) ==
pytest.approx(0.25)
tests\test_fusion_features.py:61:    assert ff.in_court_ratio(counts, min_players=1) ==
pytest.approx(0.75)
tests\test_fusion_features.py:65:    assert ff.in_court_ratio([]) == 0.0
backend\pipeline\detectors\fusion_features.py:99:def in_court_ratio(counts: Sequence[int],
min_players: int = 2) -> float:
backend\pipeline\detectors\fusion_features.py:192:    "in_court_ratio",
"shuttle_player_colocation",
backend\pipeline\detectors\fusion_features.py:208:        "in_court_ratio":
in_court_ratio(counts, min_players),
backend\pipeline\validators\relevance.py:54:        min_court_ratio =
rel_cfg.get("court_pixels_min_ratio", sport_rel_cfg.get("court_pixels_min_ratio", 0.05))
backend\pipeline\validators\relevance.py:93:            "green_threshold": min_court_ratio
backend\pipeline\validators\relevance.py:97:        if avg_green_ratio < min_court_ratio:
backend\pipeline\validators\relevance.py:101:                message=f"Video rejected: Relevance
check failed. The scene does not contain a recognizable {sport_name} court (court area found:
{round(avg_green_ratio * 100, 2)}% vs min required {round(min_court_ratio * 100, 2)}%).",
```

10. assistant (2026-06-13T19:15:20.526Z)

This confirms the design intent. The documentation (MULTI_SIGNAL_FUSION_PLAN.md:138) explicitly defines `in_court_ratio` as "fraction of frames with ?2 in-court players" ? i.e., the threshold of 2 is the *definition* of this feature column, not a tunable. It is consumed only as a persisted feature for P3's learned fusion, never in the served gating path (which uses `players_in_court`, line :124).

Let me check the experiment harness column mapping to be fully sure `in_court_ratio` isn't tied to `min_players` there.

11. user (2026-06-13T19:15:21.407Z)

```
85              `segmenter_registry` + `IndexingConfig` do the actual selection in production).
86              New cues are recorded here default-OFF.
87            - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
88              ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
89              `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
90              windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
91            - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
92              whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
93              cue persists that feature).
94            - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
95              `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
96              required for `compare_unlabeled` on a learned variant.
97            - ``feature_names`` ? the persisted features the learned filter consumes. When set
```

```
98              WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
99              (honest) ? the recipe, not a pinned artifact.
100           - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
101             overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
102         """
103
104         name: str
105         segmenter: str = "wasb_hybrid"
106         config_overrides: Dict[str, Any] = field(default_factory=dict)
107         cue_flags: Dict[str, bool] = field(default_factory=dict)
108         min_crossings: int = 0
109         min_players: int = 0
110         classifier_model: Optional[str] = None
111         feature_names: Optional[List[str]] = None
112         threshold: float = 0.5
113         merge_gap: float = 1.0
114
115         def is_learned(self) -> bool:
116             """True if this variant applies a learned classifier (pinned model or
recipe)."""
117             return self.classifier_model is not None or bool(self.feature_names)
118
119         def to_dict(self) -> dict:
120             return {
121                 "name": self.name,
122                 "segmenter": self.segmenter,
123                 "config_overrides": self.config_overrides,
124                 "cue_flags": self.cue_flags,
125                 "min_crossings": self.min_crossings,
126                 "min_players": self.min_players,
127                 "classifier_model": self.classifier_model,
128                 "feature_names": self.feature_names,
129                 "threshold": self.threshold,
130                 "merge_gap": self.merge_gap,
131             }
132
133         @classmethod
134         def from_dict(cls, d: dict) -> "Variant":
135             known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
136             return cls(**{k: v for k, v in d.items() if k in known})
137
138         def spec_hash(self) -> str:
139             """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
140             and makes a result reproducible. The pinned **control** variant always hashes
the
141             same, so a regression is always traceable to a recipe change, never to drift."""
142             blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
143             return sha1(blob.encode()).hexdigest()[:12]
144
145
146     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
147     #: filter. Always the comparison reference; "rollback" = make this the served variant.
148     CONTROL = Variant(name="control")
149
150
151     # ---------------------------------------------------------------- persisted candidates
152
153
154     @dataclass
155     class VideoCandidates:
156         """One video's persisted detection, ready for offline re-scoring.
157
158         ``intervals`` are the candidate windows; ``features`` is column-major
159         (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
160         ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
```

```
        with `load_video_candidates`, or directly in tests.
        """

        name: str
        intervals: List[Interval]
        features: Dict[str, List[float]] = field(default_factory=dict)
        gts: Optional[List[Interval]] = None


    def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
        """Persisted feature column, defaulting missing/short columns to 0.0 (matches
        `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
        col = vc.features.get(name)
        n = len(vc.intervals)
        if col is None:
            logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                           name, vc.name)
            return [0.0] * n
        if len(col) != n:
            raise ExperimentError(
                f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
        return [float(x) for x in col]


    def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
        cols = [_feature_column(vc, name) for name in feature_names]
        return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]


    def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
        """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
        n = len(vc.intervals)
        mask = [True] * n
        if variant.min_crossings > 0:
            col = _feature_column(vc, "net_crossings")
            mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
        if variant.min_players > 0:
            col = _feature_column(vc, "players_in_court")
            mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
        return mask


    def predict(variant: Variant, vc: VideoCandidates,
                model: Optional[LogisticRegression] = None) -> List[Interval]:
        """Variant prediction: gate by persisted features, optionally filter by a learned
        model, then merge. Pure and deterministic.

        ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
        fold; if omitted and the variant pins ``classifier_model``, that frozen model is
        loaded. A learned variant with neither a supplied nor a pinned model raises ? there
        is nothing to score with.
        """
        mask = _gate_mask(variant, vc)
        if variant.feature_names:
```

12. assistant (2026-06-13T19:15:32.725Z)

This fully confirms the design. The experiment harness's Gate B (`_gate_mask`, experiment.py:197-199) also gates on the `players_in_court` feature column using `variant.min_players`, NOT on `in_court_ratio`. So in BOTH the served path (`_select_for_handoff`) and the offline harness, `min_players` thresholds operate on `players_in_court`, while `in_court_ratio` remains a fixed-definition persisted feature whose internal `min_players=2` is part of its column definition (documented as "fraction of frames

with ?2 in-court players").

I have everything I need. Let me record my assessment.

Verification summary:

Factual claims all hold:
- Two distinct `min_players` exist: `FusionConfig.min_players` (models.py:102, default 0) and `compute_person_features`/`in_court_ratio`'s `min_players` (fusion_features.py:200, :99, default 2). CONFIRMED.
- `_person_features` (fusion_hybrid.py:106-107) calls `ff.compute_person_features(...)` without forwarding `min_players`, so persisted `in_court_ratio` always uses the hardcoded default 2, independent of `FusionConfig.min_players`. CONFIRMED.

But the finding is explicitly NOT a bug:
- The finding itself concedes "No functional bug today" and rates it "low".
- The two are correctly independent **by design**: `in_court_ratio`'s threshold-2 is the column's *definition* (docs/MULTI_SIGNAL_FUSION_PLAN.md:138: "fraction of frames with ?2 in-court players"), a fixed config-independent feature for P3's learned fusion.
- `FusionConfig.min_players` (Gate B) gates on `players_in_court`, not `in_court_ratio` ? verified in both the served path (fusion_hybrid.py:124) and the offline harness (experiment.py:197-199).
- So the name collision causes no incorrect behavior, no wrong persisted value, no mis-gating.

This is a documentation/readability nitpick about a name collision that the author already correctly handled. The proposed "fix" is an optional one-line comment. It does not represent a defect in the code's behavior. Per the instructions to default to `real=false` for nitpicks/non-bugs, this should be marked not-real as a code issue, while acknowledging the underlying facts are accurate.

13. user (2026-06-13T19:15:44.714Z)

Structured output provided successfully

14. assistant (2026-06-13T19:15:51.284Z)

I've completed the adversarial verification. The finding's factual claims all hold against the code, but it does not constitute a real code issue ? it is a documentation/naming nitpick that the finding itself concedes is "No functional bug today," and I've marked it `real=false` accordingly.

Key files verified:
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/fusion_hybrid.py` (lines 106-107, 117, 124)
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/fusion_features.py` (lines 99, 200, 208)
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/models.py` (line 102)
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/experiment.py` (lines 197-199)

The two `min_players` are independent **by design**: the served/offline Gate-B threshold (`FusionConfig.min_players`) gates on the `players_in_court` feature column in both the live path and the experiment harness, while `in_court_ratio`'s internal `min_players=2` is the fixed *definition* of a config-independent persisted feature column for P3's learned fusion (documented in `docs/MULTI_SIGNAL_FUSION_PLAN.md:138`). They never interact, so there is no behavioral defect.